↓↓↓スライド公開済みです↓↓↓

https://twitter.com/mathetake のTweet **or** connpassのイベントページ

Proxy-wasm and its landscape

Envoy Meetup Tokyo #2 — @mathetake

もくじ

1. イントロ
2. WASM(WebAssembly)入門
3. Proxy-wasm入門
4. Proxy-wam vs WASI(WebAssembly System Interface)
5. まとめ

1. イントロ

背景 - Envoy/Istioの拡張機構

- Envoy
  - c++のみ(+Lua)
  - 独自build or upstreamへのmerge必須
  - 再配布時にすべてのEnvoyのrolloutが必須
- Istio(≦1.4.x)
  - Mixerによるextra hop
  - 独自protocol実装不可
  - 限られた応用 (datadog adapter, prometheus adapter, …)

背景

- “Powerful tooling is nothing without a great developer experience”*
- 以下の要件を満たすような拡張機構がEnvoyに欲しい
  - C++だけなく可能な限り多くの言語をサポート
  - 拡張追加/変更時Envoyのrolloutの必要なし
  - Envoyのrelease cycleから拡張機能をdecouple可能な程の柔軟性
  - セキュアな仕様

* https://www.solo.io/blog/an-extended-and-improved-webassembly-hub-to-helps-bring-the-power-of-webassembly-to-envoy-and-istio/

背景

- “Powerful tooling is nothing without a great developer experience”*
- 以下の要件を満たすような拡張機構がEnvoyに欲しい
  - C++だけなく可能な限り多くの言語をサポート
  - 拡張追加/変更時Envoyのrolloutの必要なし
  - Envoyのrelease cycleから拡張機能をdecouple可能な程の柔軟性
  - セキュアな仕様

WASMで機能拡張というアイデア

* https://www.solo.io/blog/an-extended-and-improved-webassembly-hub-to-helps-bring-the-power-of-webassembly-to-envoy-and-istio/

2020 Posts

Introducing Workload Entries

Safely Upgrade Istio using a Canary Control Plane Deployment

Direct encrypted traffic from IBM Cloud Kubernetes Service Ingress to Istio Ingress Gateway

Extending Istio 1.5 with

Redefining extensibility in proxies - introducing WebAssembly to Envoy and Istio

A new interface for extending proxy servers allows moving Istio extensibility from the control plane into the sidecar proxies themselves

BY **CRAIG BOX, MANDAR JOG, JOHN PLEVYAK, LOUIS RYAN, PIOTR SIKORA (GOOGLE), YUVAL KOHAVI, SCOTT WEISS (SOLO.IO)** |

MARCH 5, 2020 | 8 MINUTE READ

https://istio.io/latest/blog/2020/wasm-announce/

Google Open Source Blog

The latest news from Google on open source releases, major projects, events, and student outreach programs.

WebAssembly brings extensibility to network proxies

Thursday, March 12, 2020

With the Istio 1.5 release we are happy to introduce WebAssembly (Wasm) extensions in Envoy, built with our long running collaborators Lyft and IBM. With partners like Solo.io deepening their engagement as well, we are excited to see the community and ecosystem developing around this segment of the open source world.

https://opensource.googleblog.com/2020/03/webassembly-brings-extensibility-to.html

Proxy-wasmとは

- WebAssembly(WASM) for Proxiesの略称
- Istio 1.5.0のリリースとともに発表
- WASMとホスト環境(proxy)の間のABI(Application Binary Interface)
  - リファレンス実装としてのEnvoy
  - V8エンジンがまるっとEnvoyの中で動いてる
- WASMモジュールをproxyの中で動かして拡張するための仕様
- status: Alpha

なぜWASMか

- 多くの言語(30+)で開発可能
- nativeに近いスピード
- W3 foundationによるサポート
- toolingの充実
- Security orientedなランタイム/仕様

Proxy-wasmの開発

- developerがProxy-wasmのABIに準拠したWASMバイナリを作る
  - 各言語のsdkを使って実装する
- 当然developerはWASMのことを意識してコードを書く
  - (通常の)システムコールは呼べないとかそういう基本的な事
- Proxy-wasmと仲良くなるにはWASM自体の理解が必須
  - WASMに入門しましょう

2. WASM入門

WASM(WebAssembly)とは

- WASM = Stack machineとそのバイトコードの仕様
  - Portable
  - At near-native speed
  - 複数言語からのコンパイルが可能
  - Text形式(wat)も存在 —> 便利!
- 元々はウェブサイト(js)の高速化が目的
- ウェブ標準認定: https://www.w3.org/2019/12/pressrelease-wasm-rec.html
- 仕様書: https://webassembly.github.io/spec/core/intro/index.html

WASMとHost環境

- Specificationはhost環境(VM自体の実行環境)への要求を持たない
  - 唯一存在するのは **host functions** という概念
    - WASM側から自由に呼び出せるhost環境が提供する関数
    - I/Fはhostの実装者が自由に策定
    - Importの仕様の一部
- (Host環境も含め)外部からの呼び出しを想定したexport directiveも存在
  - WASMの中の関数をHostや他のWASMから呼び出すことも可能

WASMとHost環境

- Specificationはhost環境(VM自体の実行環境)への要求を持たない
  - 唯一存在するのは **host functions** という概念
    - WASM側から自由に呼び出せるhost環境が提供する関数
    - I/Fはhostの実装者が自由に策定
    - Importの仕様の一部
- (Host環境も含め)外部からの呼び出しを想定したexport directiveも存在
  - WASMの中の関数をHostや他のWASMから呼び出すことも可能

ホスト環境の実装自由度高

WASMとHost環境

```go
package main

func main() {
  hostFunc()
}

//export malloc
func malloc(size uint32) *byte {
  v := make([]byte, size)
  return &v[0]
}

func helloFromWASM()
```

WASMとHost環境

```
package main

func main() {
  hostFunc()
}
```

exportされた関数 -> host環境から呼び出し可能

```
//export malloc
func malloc(size uint32) *byte {
  v := make([]byte, size)
  return &v[0]
}
```

```
func helloFromWASM()
```

WASMとHost環境

```
package main

func main() {
  hostFunc()
}
```

exportされた関数 -> host環境から呼び出し可能

```
//export malloc
func malloc(size uint32) *byte {
  v := make([]byte, size)
  return &v[0]
}
```

```
func helloFromWASM()
```

Bodyがない関数 -> hostによる実装を期待

WASMとHost環境

```
package main

func main() {
  hostFunc()
}
```

WASMのテキスト表現

```
//export malloc
func malloc(size uint32) *byte {
  v := make([]byte, size)
  return &v[0]
}
```

```
(func $malloc (export "malloc")…
```

```
func helloFromWASM()
```

```
(import "env" “helloFromWASM” (…))
```

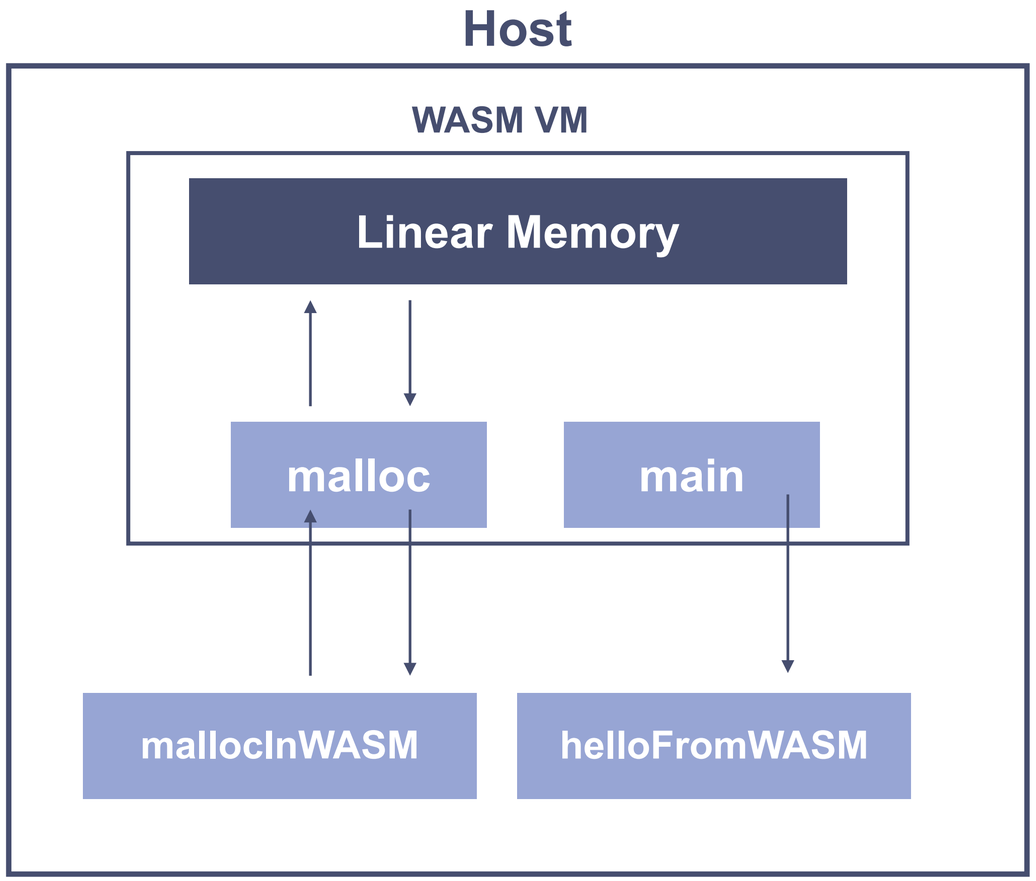
WASMとHost環境

```go
package main

func main() {
  hostFunc()
}

//export malloc
func malloc(size uint32) *byte {
  v := make([]byte, size)
  return &v[0]
}

func helloFromWASM()
```



https://hacks.mozilla.org/2017/02/where-is-webassembly-now-and-whats-next/

WASMとセキュリティ

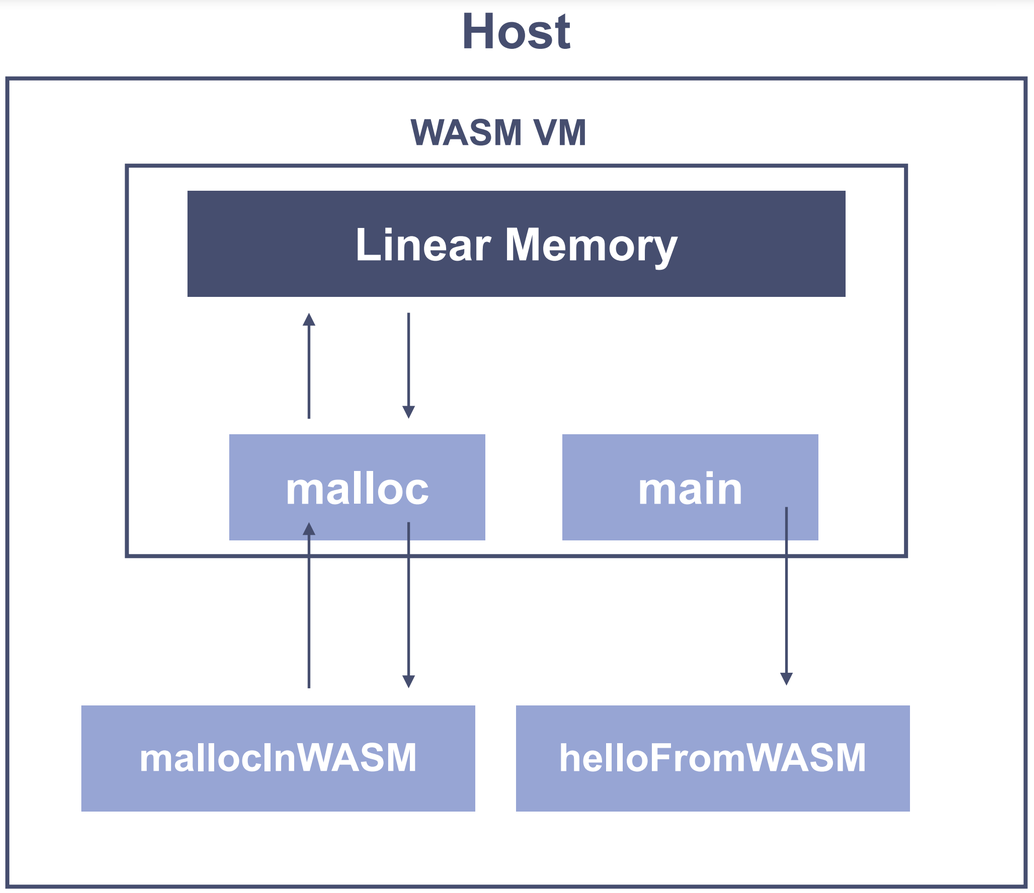
- WASMの実行自体はHost環境から切り離されたVM実行される
  - ホストに直接アクセスは不可
    - 事前にHostが定めたImportされた関数(ABI,API)を介してお話できるだけ
- loadのタイミングですべての関数がdeclareされてなければならない
  - 先程の例: helloFromWASMはprogram開始時にdeclareされてなければいけない
- Runtimeにすべての分岐命令をvalidate
- Memory Safety: Hostの中の独立したBuffer —> Leak耐性も

3. Proxy-wasm入門

Proxy-wasmとは(もう一度)

- WebAssembly(WASM) for Proxiesの略称
- Istio 1.5のリリースとともに発表
- **<u>WASMとホスト環境(proxy)の間のABI(Application Binary Interface)</u>**
  - リファレンス実装としてのEnvoy
  - V8エンジンがまるっとEnvoyの中で動いてる
- WASMモジュールをproxyの中で動かして拡張するための仕様

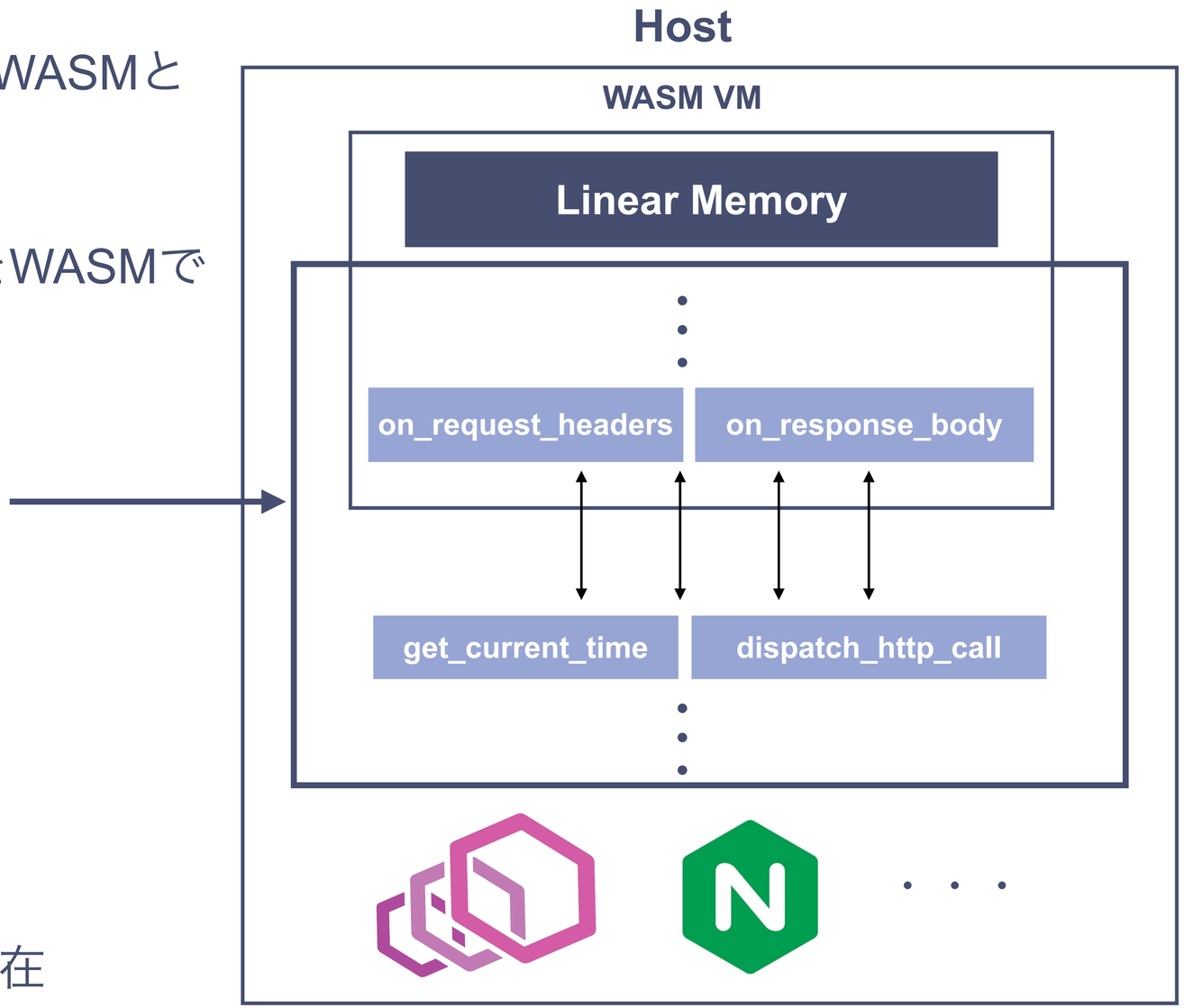
Proxy-wasmとは



https://hacks.mozilla.org/2017/02/where-is-webassembly-now-and-whats-next/

Proxy-wasmとは

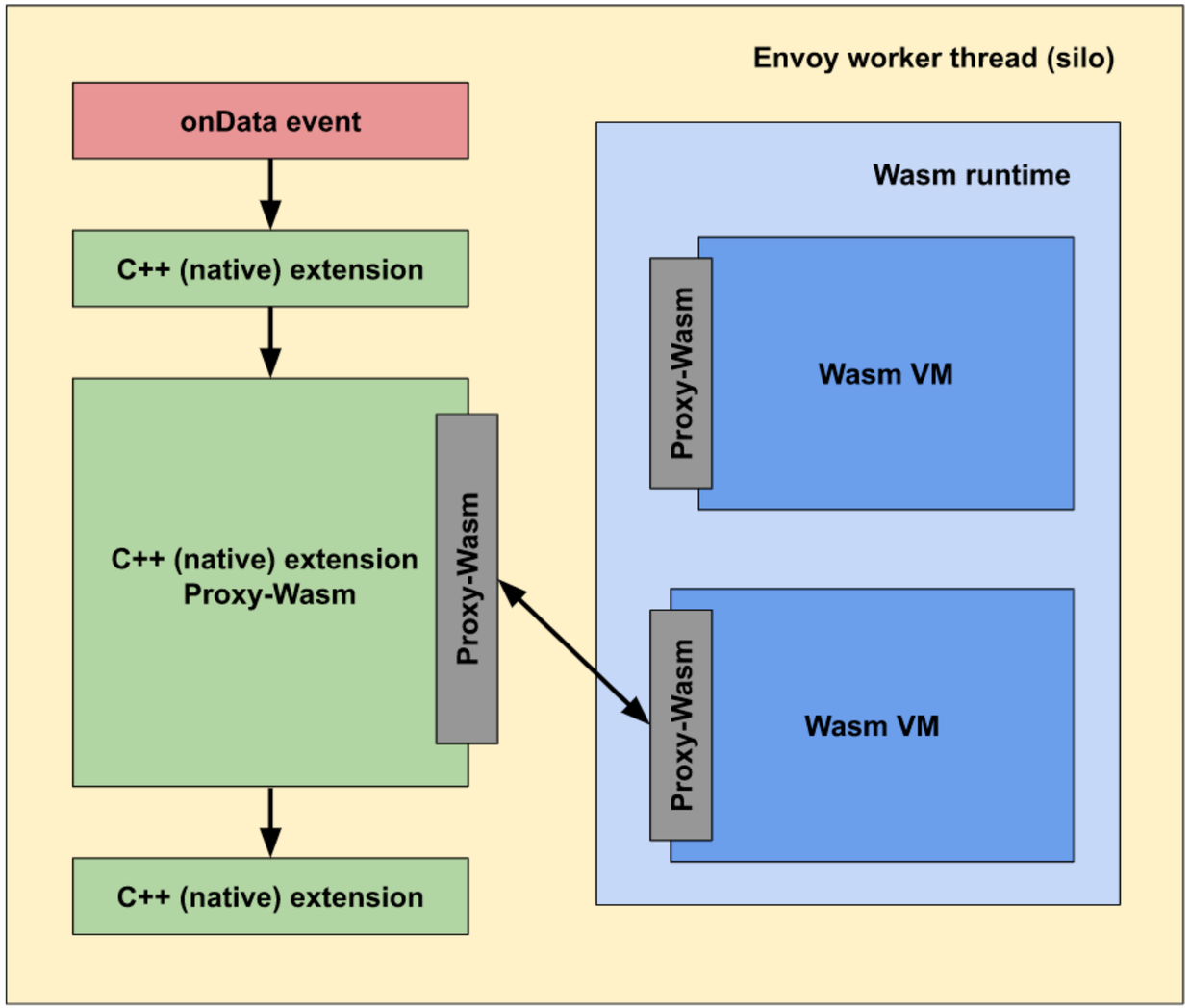
- proxyサーバーへの埋め込みを前提としたWASMとホスト環境の間の仕様の一つ
- リクエストのライフサイクルのイベントをWASMでhandlingするためのI/F
- 仕様そのものはEnvoyから独立
  - そのリファレンス実装としてEnvoy
- github.com/proxy-wasm が主導
  - 仕様: github.com/proxy-wasm/spec
- c++, rust, go, AssemblyScript向けsdkが存在



https://twitter.com/nginx

Proxy-wasm in Envoy

- 1 (VM, WASM module) / 1 worker
- バイナリはdiskかxDS経由でload
- Fork先で開発: envoyproxy/envoy-wasm
  - Upstreamへはまだマージされていない
  - シュッと試したい人はistioのimageが楽: istio/proxyv2
- 通常のfilterと同じように設定
  - API: envoyproxy/envoy-wasm/api/envoy/extensions/wasm/v3



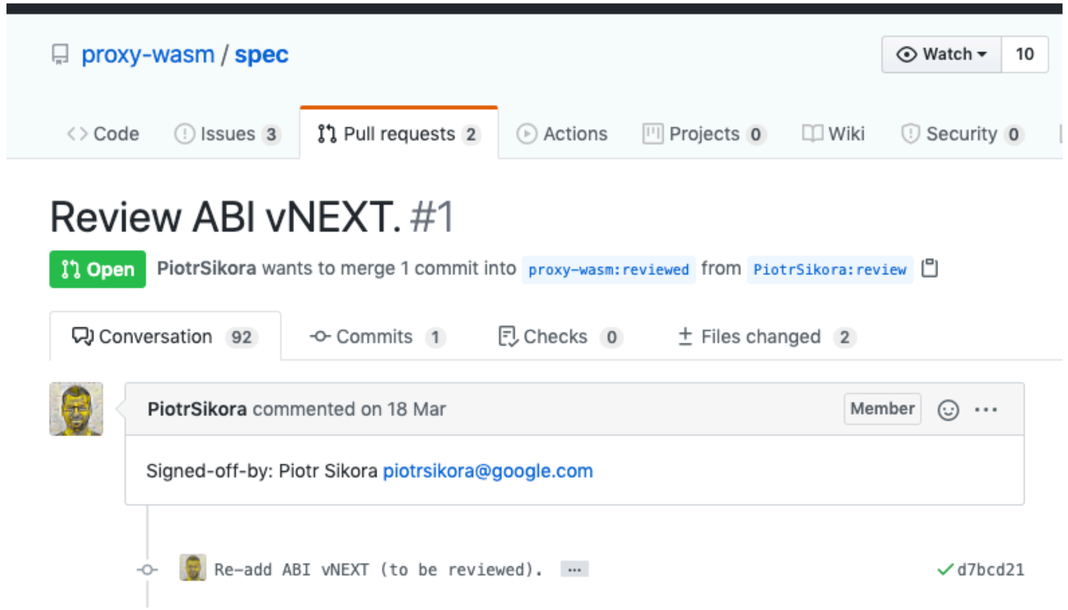
Proxy-wasm in Envoy

- 1 (VM, WASM module) / 1 worker
- バイナリはdiskかxDS経由でload
- Fork先で開発: envoyproxy/envoy-wasm
  - Upstreamへはまだマージされていない
  - シュッと試したい人はistioのimageが楽: istio/proxyv2
- 通常のfilterと同じように設定
  - API: envoyproxy/envoy-wasm/api/envoy/extensions/wasm/v3

```yaml
http_filters:
  - name: envoy.filters.http.wasm
    config:
      config:
        name: "auth"
        root_id: "auth"
        configuration: "auth_service"
        vm_config:
          vm_id: "my_vm_id"
          runtime: "envoy.wasm.runtime.v8"
          code:
            local:
              filename: build/optimized.wasm
  - name: envoy.router
    config: {}
```

ABI overview

- github.com/proxy-wasm/spec/tree/master/abi-versions/vNEXT
  - 絶賛under review github.com/proxy-wasm/spec/pull/1
  - ~~(これがマージされないと色々と進まない…)~~
- 大きく分けて二種類
  - Hostでの実装を期待する関数
  - WASM側で実装すべき関数(=拡張そのもの)

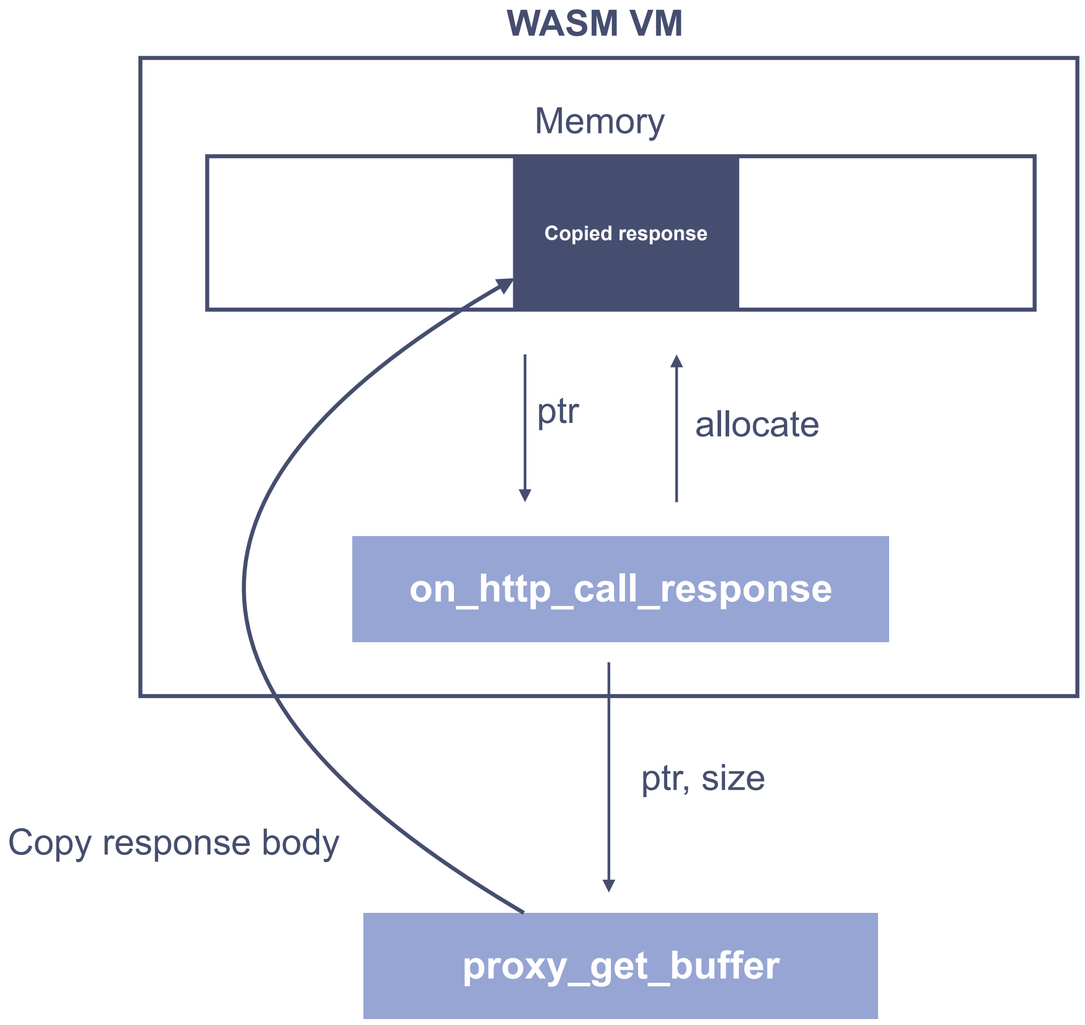


ABI overview - Host関数

- proxy_dispatch_{http,grpc}_call
  - WASM側から任意のClusterに対してリクエストをinvoke
  - 認証サーバに問い合わせるとかそういうのに使える
- proxy_{define,get,record,increment}_metric
  - WASM側からメトリクスをいじる用
- proxy_log
  - ロギングのための関数

ABI overview - Host関数

- WASMからHost上のデータを取得する際の挙動
  - VMの中のmemoryにデータをすべてコピー
- 例: proxy_get_buffer
  - Request Bodyを取得したりする際に使用
  - return_buffer_data, return_buffer_sizeを指定
- WASM側から直接データを操作することはない



ABI overview - WASM関数

- L7
  - proxy_on_http_{request,response}_{headers, body}
    - httpリクエスト/レスポンスをハンドリング
- L4
  - proxy_on_{up,down}_stream_close
    - upstream/downstreamのコネクションcloseをハンドリング
  - proxy_on_{up,down}stream_data
    - upstream/downstreamのdata chunkをハンドリング
- proxy_set_tick_period, proxy_on_tick
  - proxy_set_tick_periodによりsetしたduration毎に proxy_on_tickが定期的に呼ばれる

Example

- OnHttpRequestHeaders
  - リクエストヘッダーが来たときに呼ばれる
  - 中でDispatchHttpCallを呼ぶ
- OnHttpCallResponse
  - Dispatchしたリクエストの返答が来たらで呼ばれる
  - そのBodyを取得してハッシュを計算
  - ハッシュの値によって403を返す
- sdkによってi/fは多少違うができることは一緒(当たり前)

```go
func (ctx *httpHeaders) OnHttpRequestHeaders(int) runtime.Action {
   ctx.DispatchHttpCall("httpbin", hs, "", [][2]string{}, 50000)
   return runtime.ActionPause
}
```

```go
func (ctx *httpHeaders) OnHttpCallResponse(
   _ uint32, _, bodySize, _ int,
) {
   b, st := ctx.GetHttpCallResponseBody(0, bodySize)
   if st != runtime.StatusOk {
      ctx.ResumeHttpRequest()
      return
   }

   s := fnv.New32a()
   if _, err := s.Write(b); err != nil {
      ctx.ResumeHttpRequest()
      return
   }

   if s.Sum32()%2 == 0 {
      runtime.LogInfo("access granted")
      ctx.ResumeHttpRequest()
      return
   }

   ctx.SendHttpResponse(403, [][2]string{
      {"powered-by", "proxy-wasm-go!!"},
   }, "access forbidden")
}
```

https://github.com/mathetake/proxy-wasm-go/blob/f1002f9c92f62fc5e5b897c81c44da4c9c7a6fea/examples/http_auth_random/main.go#L25-L66

展望

- 任意のenvoyのfilterが自分の好きな言語でかける！
  - それがenvoyだけではなくて他のproxyでも動く！
- いままで任意の言語で実装してたAPIサーバの機能をenvoyに埋め込める！
- WebAssemblyHubが大流行！ https://webassemblyhub.io/
  - 無限のenvoyのfilterを一瞬で試せる！
- Proxy-wasm nativeなプロキシサーバが登場！
- 任意のservice meshでprxoy-wasmの拡張が使える！

現状と課題

- 現状APIが全くstableではない
  - 実際 github.com/proxy-wasm/spec とsdkのAPIがかなり違う
  - envoyへのupstreamはまだ先らしい(要出典)
- istioのTelemetry v2に導入されてるが、デフォルトはnull vmを使ってる:
  - wasmフィルターなんだけどnative compiled —> WASMのVMは動いてない
- カスタムプロトコルの実装は不可能
  - 例えばredis filterみたいなものは実装できない
- データのコピー等によるパフォーマンスの問題
  - GC付きの言語からだと言語処理系レベルでのチューニングが求められそう…?

現状と課題

Tetrate
@Tetrateio

#TetrateAMA @mattklein123 WebAssembly should be upstream in the next month or two.

Tetrate
@Tetrateio

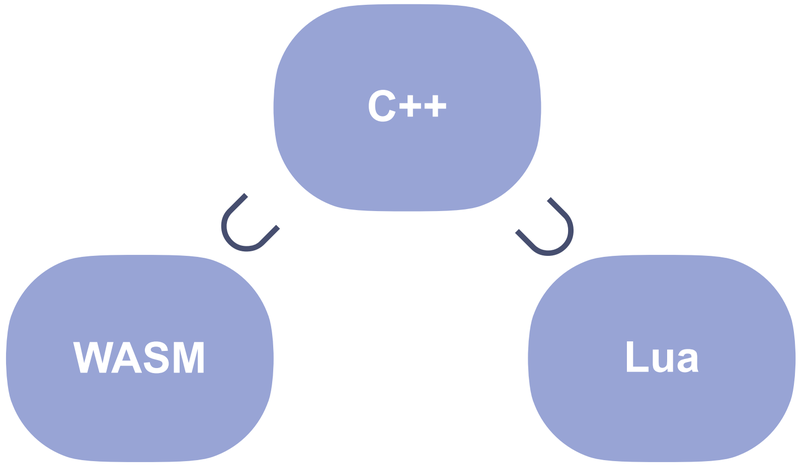
"It's really early days. I'm very optimistic about web assembly as the future of the way that we'll extend Envoy, but realistically, it's goign to take multiple years to reach maturity, and the SDKs, the runtimes and the overall ecosystem."

3:58 AM · Jun 10, 2020 · Twitter Web App

他の拡張法との差分

Taegyun Kim 2:26 AM
Is there anything that's only possible with WASM extension filters? compared to existing c++/lua extension filters?

John Plevyak 3:03 AM
@Taegyun Kim The WASM extensions have a couple additional APIs, but generally they are a subset of what you do with C++ extensions. Lua extensions are a separate issue as they are a completely separate implementation although again are a subset of the standard C++ extensions. Do be a aware that general C++ extensions are not as well encapsulated, so are going to be more likely to break going forward (after we get the Wasm API upstreamed which is going to have stricter versioning).

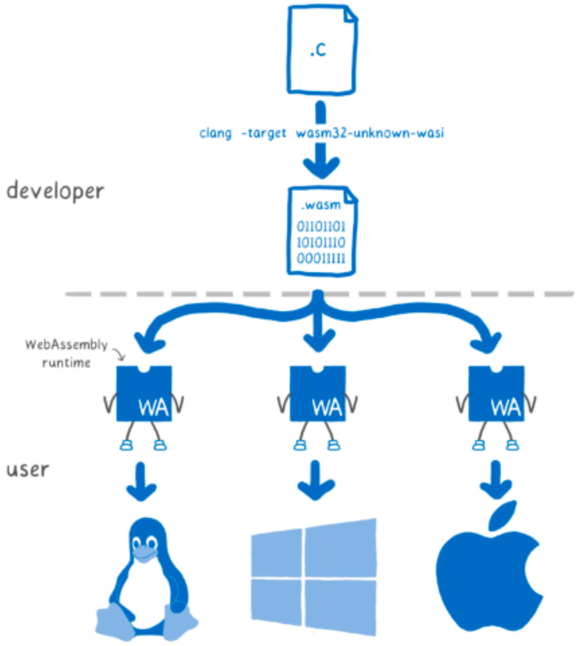


https://envoyproxy.slack.com/archives/CM2CQ4XJ5/p1590516380088800

4. Proxy-wasm vs WASI

WASI とは

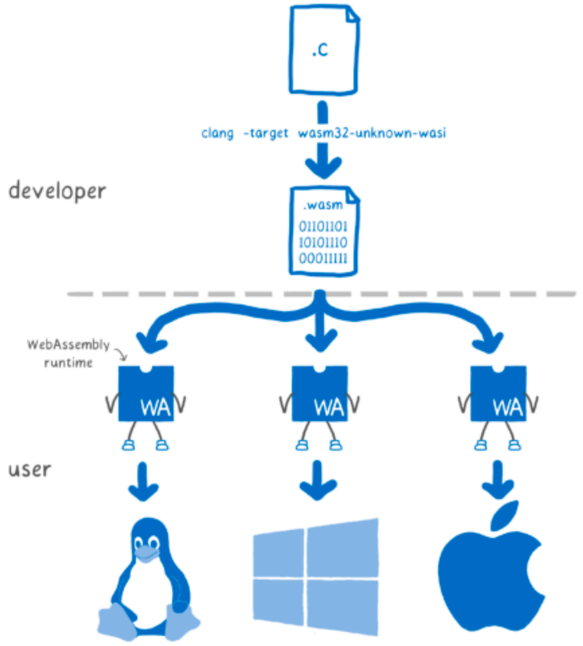
- WASI = WebAssembly System Interface
- WASMのHost環境が提供するAPI仕様
- motivation: WASMでunix likeなsyscall呼びたい
  - ソケットとかfilesystemとかそういうの扱いたい
- 最近話題になったkrusletとかもWASIベース
  - github.com/deislabs/krustlet
- 2019/03に発表(NEW!)



https://hacks.mozilla.org/2019/03/standardizing-wasi-a-webassembly-system-interface/

WASI とは

- WASI = WebAssembly System Interface
- **WASMのHost環境が提供するAPI仕様**
- motivation: WASMでunix likeなsyscall呼びたい
  - ソケットとかfilesystemとかそういうの扱いたい
- 最近話題になったkrusletとかもWASIベース
  - github.com/deislabs/krustlet
- 2019/03に発表(NEW!)



https://hacks.mozilla.org/2019/03/standardizing-wasi-a-webassembly-system-interface/

Proxy-wasm vs WASI

- どちらも全く同じ土俵
  - WASMとホスト環境の間のABI
- 実際に共通する関数もある
  - proxy_get_current_time
  - __wasi_clock_time_get
- 将来的にはWASIへのマージを期待

What is the relationship between WASI and proxy-wasm?

Asked 2 months ago Active 2 months ago Viewed 124 times

0

WASI and proxy-wasm seem to have similar goals - defining an ABI to allow interaction of WASM modules with the host environment (i.e. supporting syscalls), with proxy-wasm focussing on modules that are embedded into proxies.

What's the relationship (current or planned) between the two? I've seen this but not sure what was covered.

webassembly envoyproxy wasi

share improve this question follow edited Apr 1 at 11:24

asked Apr 1 at 10:59
dippynark
1,630 ● 7 ● 17

add a comment

1 Answer

Active Oldest Votes

2

proxy-wasm is/was used for the initial development, in order to bootstrap the ecosystem, and to gauge the interest of other vendors in the industry (which was a great success, since others have already created: AssemblyScript SDK, TinyGo SDK, and WebAssembly Hub).

But the idea was always to contribute this ABI to WASI, and we spoke at the WASI meetups on 2019/10/15 and 2020/03/26 about it, so expect the ABI to be incorporated into WASI over the next few weeks/months.

share improve this answer follow

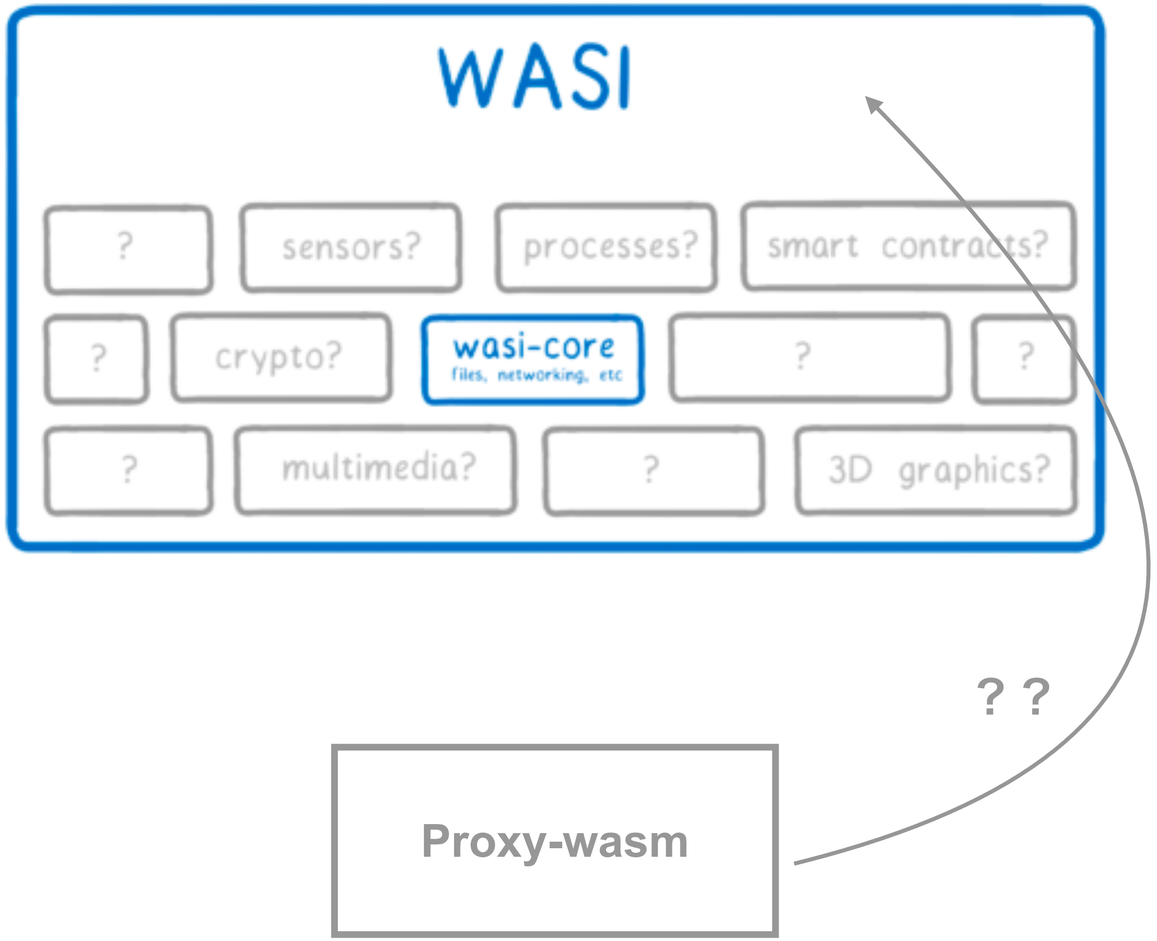
answered Apr 2 at 16:05
P Piotr Sikora
36 ● 1

add a comment

https://stackoverflow.com/questions/60969344/what-is-the-relationship-between-wasi-and-proxy-wasm

Proxy-wasm vs WASI

- どちらも全く同じ土俵
  - WASMとホスト環境の間のABI
- 実際に共通する関数もある
  - proxy_get_current_time
  - __wasi_clock_time_get
- 将来的にはWASIへのマージを期待



https://hacks.mozilla.org/2019/03/standardizing-wasi-a-webassembly-system-interface/

WASI とは - 例

- fd_write — writev的なもの
- TinyGoではpanicキーワードの実装にfd_writeを使っている
  - fd_write経由でホストにpanicのメッセージを出力
  - tinygo/runtime_wasm.go
- 他にもそれっぽいのがたくさん

`fd_write(fd: fd, iovs: ciovec_array) -> (errno, size)`

Write to a file descriptor. Note: This is similar to `writev` in POSIX.

Params

- `fd` : `fd`
- `iovs` : `ciovec_array` List of scatter/gather vectors from which to retrieve data.

Results

- `error` : `errno`
- `nwritten` : `size` The number of bytes written.

https://github.com/WebAssembly/WASI/blob/master/phases/snapshot/docs.md#-fd_writefd-fd-iovs-ciovec_array---errno-size

まとめ

まとめ

- WASM(WebAssembly)はportableなバイナリフォーマット&VM仕様
  - ブラウザだけではなくて、様々なところに組み込みが可能
- Proxy-wasmはWASM/ホスト環境間のABI仕様
  - プロキシサーバへの埋め込みを想定 —> リファレンス実装としてのEnvoy
  - イベントをWASMの中でhandling
  - 現状できることは限られているが可能性は無限大
- Proxy-wasmはWASI(WebAssembly System Interface)と仲良し
  - いつの日かWASIファミリーに仲間入りするかもしれない

References

- https://webassembly.org/docs/security/
- https://istio.io/blog/2020/wasm-announce/
- https://www.solo.io/blog/an-extended-and-improved-webassembly-hub-to-helps-bring-the-power-of-webassembly-to-envoy-and-istio/
- https://opensource.googleblog.com/2020/03/webassembly-brings-extensibility-to.html
- https://hacks.mozilla.org/2019/03/standardizing-wasi-a-webassembly-system-interface/
- https://hacks.mozilla.org/2017/07/memory-in-webassembly-and-why-its-safer-than-you-think/
- https://github.com/solo-io/wasme
- https://github.com/proxy-wasm/spec
- https://github.com/proxy-wasm/proxy-wasm-rust-sdk
- https://github.com/proxy-wasm/proxy-wasm-cpp-sdk
- https://github.com/mathetake/proxy-wasm-go